

COMP3069 Computer Graphics

Lab 7

Originally prepared by Dr Kristian Spoerer of UNUK
Modified by Prof Sean He of UNNC

Learning Outcomes

After Lab 7, you are expected to be able to

1. Use a provided `Camera` data structure and library function `lookAt()` in your implementation of a virtual camera.
2. Use GLFW to check for key presses.
3. Implement camera code based on the technical definition in the lecture notes.
4. Apply the Cross-Product operator to 2 vectors and demonstrate understanding of the application.
5. Calculate points on the unit sphere using the trigonometric functions and two rotation values.
6. Remember to convert angles to radians when using the trigonometric functions.
7. Demonstrate understanding, implementation, and use of a virtual camera.
8. Write a callback function (Optional part B).

Part A Set up

In the COMP3069 directory, create a new project called

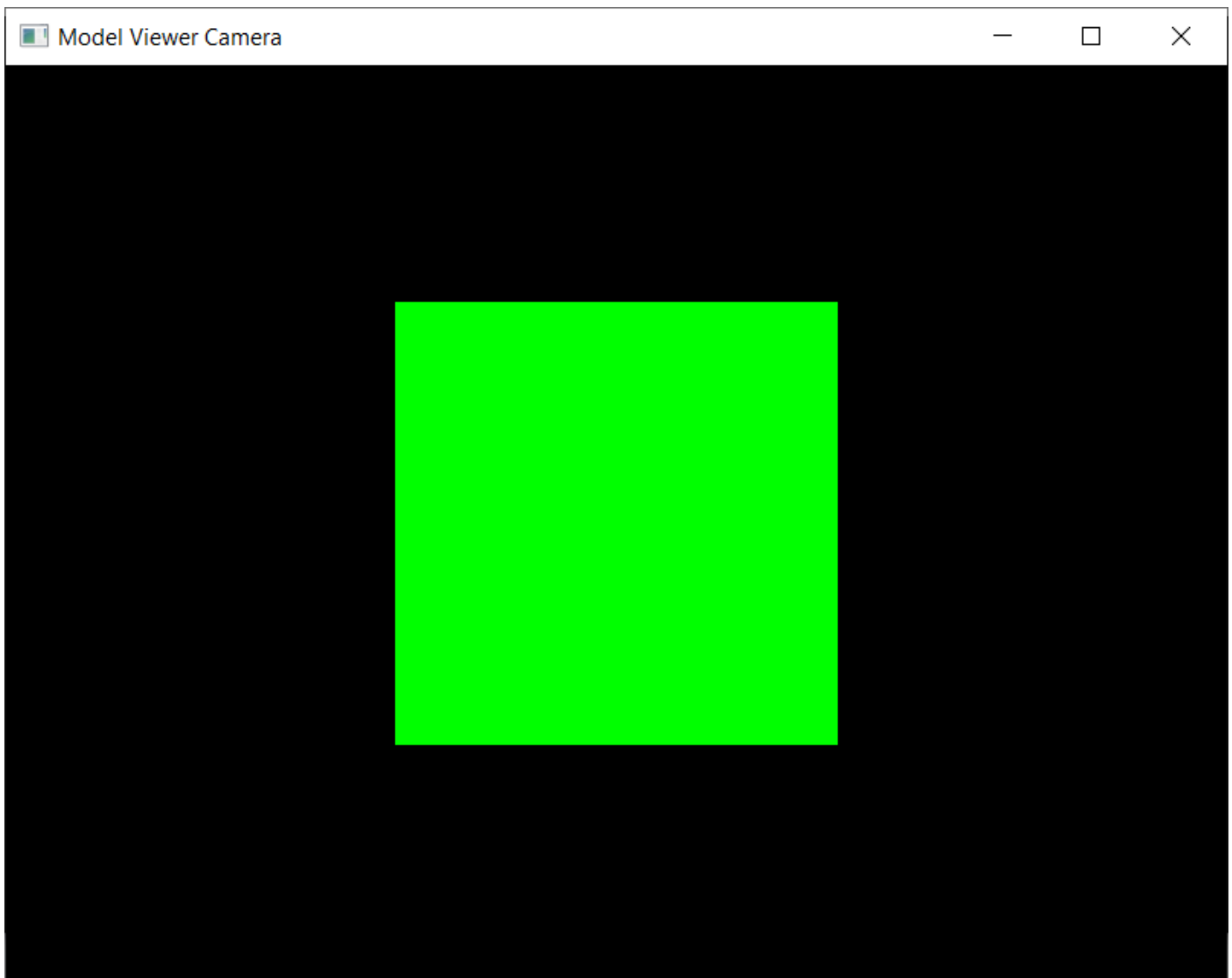
Lab7a

Download Lab7a.cpp from Moodle and copy the code to the Lab7a.cpp in your new project.

Download the vertex shader mvp.vert and fragment shader col.frag from Moodle, copy them to your project, and add them to the Source Files in the Solution Explorer.

Download camera.h and ModelViewerCamera.h from Moodle copy them to your project, and add them to the Header Files in the Solution Explorer.

If you run the program, then the following window should appear



There is a cube at the origin $(0, 0, 0)$ with multicoloured faces. The green face of the cube is pointing at the camera, which is positioned at $(0, 0, 3)$.

In this part of the lab, we are going to implement a Model-Viewer Camera, as it is defined in the lecture notes.

The camera data structure

In Lab7a.cpp on line 66, you will see that there is a camera object of type SCamera.

We are going to use a camera data structure for implementing camera types, which you will find in camera.h.

There is an `enum` setup to use for moving the camera.

There are structure members that we will use to implement the two parts of this lab, including

1. A Position in 3D space (3D vector)
2. A Front Direction, (3D vector)
3. A Right Direction, (3D vector)
4. An Up Direction, (3D vector)
5. A Pitch and Yaw value, (float)

In Lab7a.cpp on line 89, you will see that the camera is being initialised. In camera.h there is also an `InitCamera()` function which we can use to initialise the camera

```
void InitCamera(SCamera cam)
```

`cam` is the camera that needs to be initialised.

LookAt() function

GLM provides a useful function `lookAt()` which constructs a view matrix from a camera position, target, and camera up.

```
mat4 lookAt (vec3 position, vec3 target, vec3 up)
```

`position` is the position of the camera in world space, `target` is the position of the target of the camera (i.e. a point in world space at which it is focusing), and `up` is a normalized vector which defines the camera up direction to orient the camera.

Essentially, the view matrix that is constructed can be used to translate all of the vertices in the scene to the view space so the camera `position` is at the origin, and will rotate all the vertices relative to the `target` and `up`. This is an ordinary view matrix which we have seen in chapter 6, but it has been constructed using `lookAt()`.

Model-Viewer Camera

For the first part of the lab, we are going to implement a Model-Viewer Camera, as it is defined in the lecture notes.

Implementing the Model-Viewer Camera involves two parts, i) implementing the camera code and ii) implementing the input processing code.

When the user presses a key, the camera is going to be moved on the imaginary sphere surrounding the model. This means the camera will be repositioned and reoriented at the same time.

In `ModelViewerCamera.h`, you will find the function `MoveAndOrientCamera()` which you can call every time the user presses a key to move the camera.

```
void MoveAndOrientCamera(Camera cam, vec3 target, float distance, float xoffset, float yoffset)
```

where `cam` is the camera object, `target` is the position in world space that should be the focus of the camera, `distance` is the distance from the camera to the focus target, `xoffset` and `yoffset` are the amount to move the camera around the imaginary sphere.

In Lab7a.cpp, you will find the function processKeyboard() which is going to detect the user pressing a key and call the MoveAndOrientCamera() function.

In processKeyboard(), add some code to detect if the user pressed the RIGHT arrow key and calls the MoveAndOrientCamera() function with an x_offset of +1.

```
76     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
77         glfwSetWindowShouldClose(window, true);
78
79     bool cam_changed = false;
80     float x = 0.f, y = 0.f;
81     if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS)
82     {
83         x = 1.f;
84         y = 0.f;
85         cam_changed = true;
86     }
87     if (cam_changed)
88     {
89         MoveAndOrientCamera(Camera, cube_pos, cam_dist, x, y);
90     }
```

Then, in ModelViewerCamera.h, implement the changes that need to be made to the camera position and orientation

The basic outline is

1. Update Camera Yaw and Pitch
2. Calculate the position on the imaginary sphere
3. Update the Front, Right and Up vectors

First, we need to modify the camera Yaw using the xoffset, and the camera Pitch using the yoffset, but since the camera is going to focus on the target and move and orient itself around the target, the change to the Yaw and Pitch are inverted.

```
9 void MoveAndOrientCamera(SCamera& in, glm::vec3 target, float distance, float xoffset, float yoffset)
10 {
11     in.Yaw -= xoffset;
12     in.Pitch -= yoffset;
13 }
```

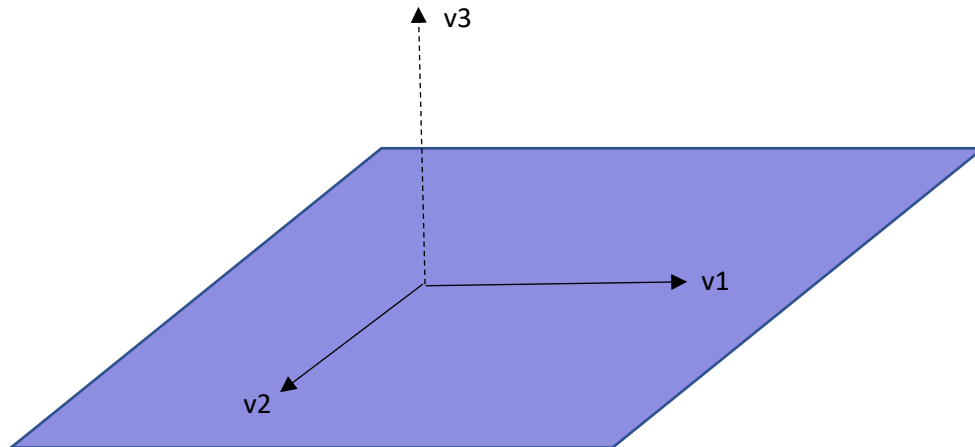
Then, we need to calculate the camera position around the outside of the imaginary sphere, as defined in the lecture notes, using the new updated Yaw and Pitch values. Make sure you convert the Yaw and Pitch to radians when using them.

After we have a new camera position, we need to calculate a new Front direction vector for the camera using the new position. The new Front vector is going to point at the target. We subtract the camera position from the target position to get a vector from the camera position to the target location. Then, we need to normalise that vector to get the direction from the camera to position the target location.

```
18 in.Front = glm::normalize(target - in.Position);
```

We need to calculate a new Right direction vector for the camera. The new Right vector is going to point at a tangent to the imaginary sphere, along the equator of the sphere. We can use cross product to calculate the new Right vector.

Recall from chapter 4 that if we have two vectors on a plane, v_1 and v_2 , then the cross product will calculate the vector, v_3 , which is perpendicular to the plane.



We can use the Camera Front vector pointing at the target, along with an absolute up vector (0, 1, 0), and calculate the dot product, and we will get the vector pointing to the Camera Right, relative to the target, which in our case is the centre of the sphere. Then, we need to normalise this calculated Camera Right vector to get the direction from the camera to the tangent on the sphere.

```
19 | | | in.Right = glm::normalize(glm::cross(in.Front, in.WorldUp));
```

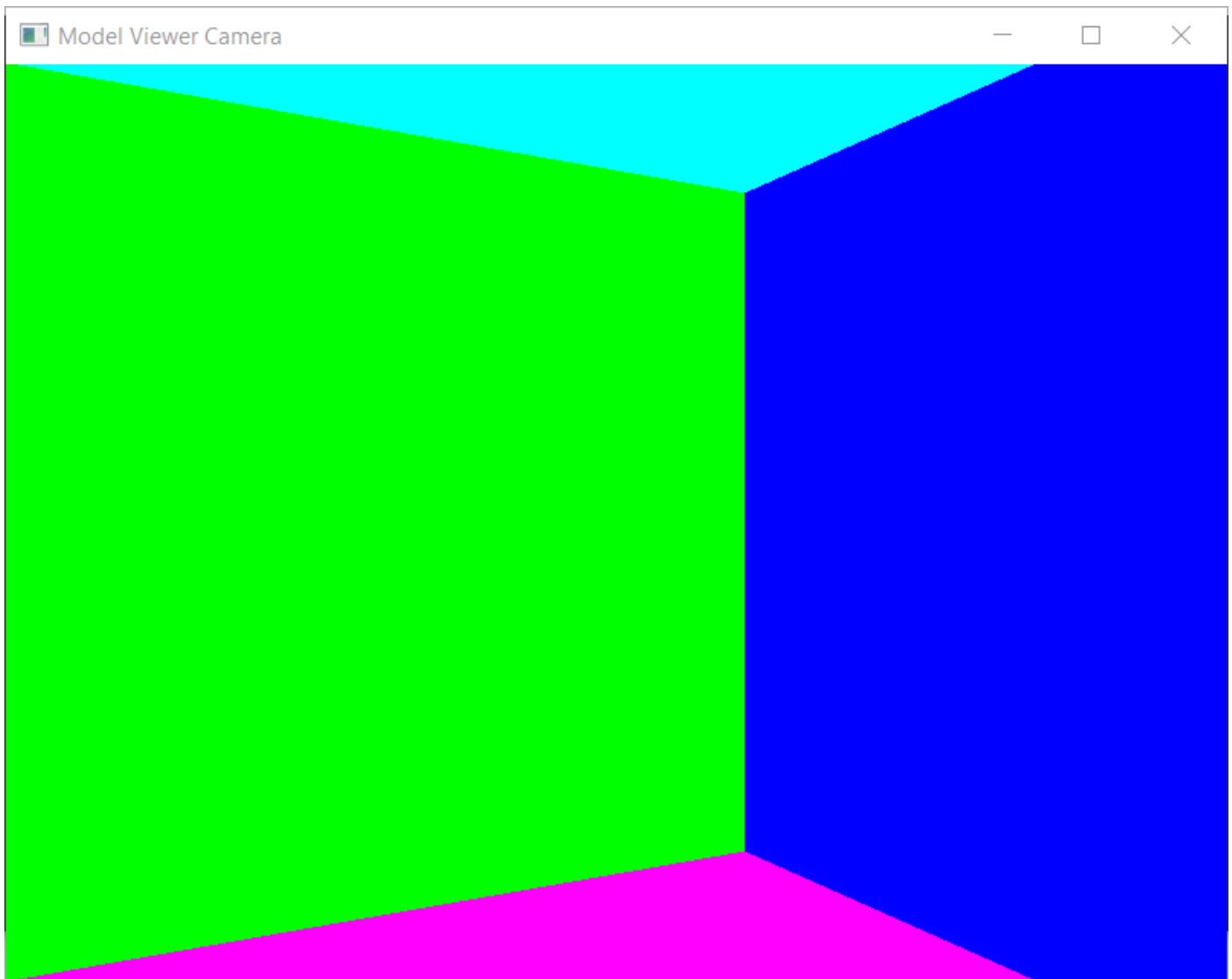
We also need to calculate a new Up direction vector for the camera. The new Up vector is going to point at a tangent to the imaginary sphere, towards the top pole, perpendicular to the Right Vector. We can use cross product to calculate the new Up vector. If we use the Camera Right vector with the camera Front vector, and calculate the dot product, then we will get the vector pointing Up towards the pole of the sphere, relative to the target. We need to normalise the Up vector to get the direction from the camera to the pole of the sphere.

```
20 | | | in.Up = glm::normalize(glm::cross(in.Right, in.Front));
```

Finally, in the render loop, where we generate a view matrix, we need to call `lookAt()` using the camera position, target, and up direction.

```
145 | | | glm::mat4 view = glm::mat4(1.f);
146 | | | view = glm::lookAt(Camera.Position, Camera.Position + Camera.Front, Camera.Up);
147 | | | glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
```

If you run the program now, you will see something like this



You can modify the x and y offset values by the camera MovementSpeed member to make the camera move faster or slower.

```
9 void MoveAndOrientCamera(SCamera& in, glm::vec3 target, float distance, float xoffset, float yoffset)
10 {
11     in.Yaw -= xoffset * in.MovementSpeed;
12     in.Pitch -= yoffset * in.MovementSpeed;
```

And modify the camera MovementSpeed in camera.h

```
24 const float MovementSpeed = .5f;
25 float MouseSensitivity = 1.f;
```

When you run the program, the camera is at the origin of world space (0, 0, 0) and if you press the RIGHT arrow key on the keyboard the camera should rotate around to the right.

Now, we need to give the camera some distance from the centre of the imaginary sphere surrounding our target point.

Use 'R' for forwards and 'F' for backwards to make the camera move forwards and backwards relative to the model being viewed. Add the code to processKeyboard() to detect the user pressing the key to move backwards, so that the distance can be modified and the MoveAndOrientCamera() function called, like this

```

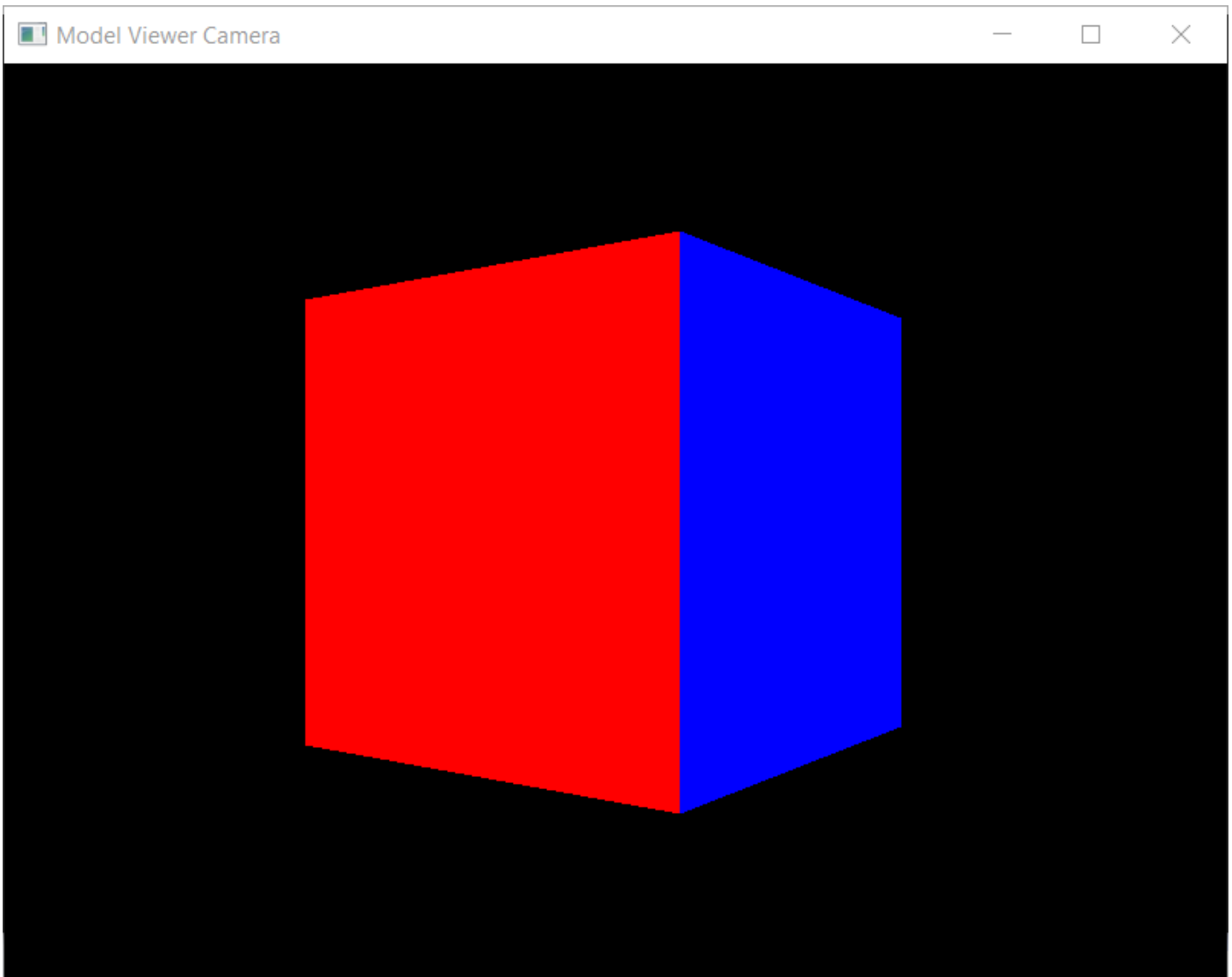
87     if (glfwGetKey(window, GLFW_KEY_F) == GLFW_PRESS)
88     {
89         cam_dist += 0.1;
90         cam_changed = true;
91     }

```

Note that the variable `cam_dist` is declared in `ModelViewerCamera.h`.

Now, when you calculate the camera Position on the unit sphere in the `MoveAndOrientCamera()` function, using the formulae in the lecture notes chapter 8, you can multiply the position vector by the newly updated `cam_dist` value.

Now, if you run the program it will look like this



You can use the RIGHT arrow key to rotate the camera around an imaginary sphere surrounding the cube, and also use the 'F' key to move further away from the cube.

Now, finish off the code in `processKeyboard()` so that the four direction keys are handled (UP, DOWN, LEFT, RIGHT), and also 'R' for forwards and 'F' for backwards to make the camera move forwards and backwards.

Now, if you run the program you should be able to rotate LEFT and RIGHT around the sphere, and move closer and further away. Modify `MovementSpeed` in `camera.h` until you're happy with the speed.

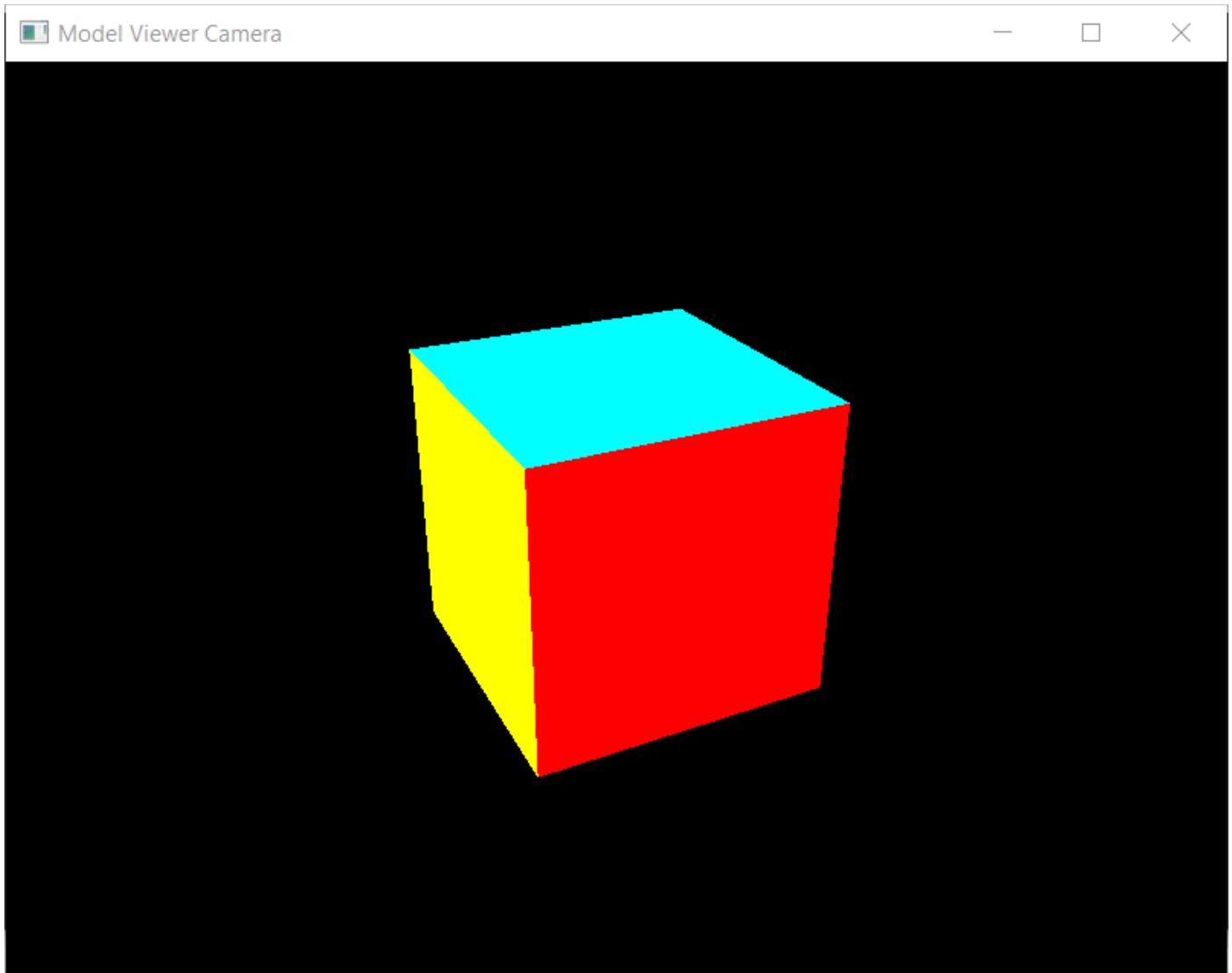
If you rotate UP and DOWN, you will notice that the rotation gets strange at the two poles of the sphere. Fix that by limiting the Camera Pitch to between -89 and 89.

```

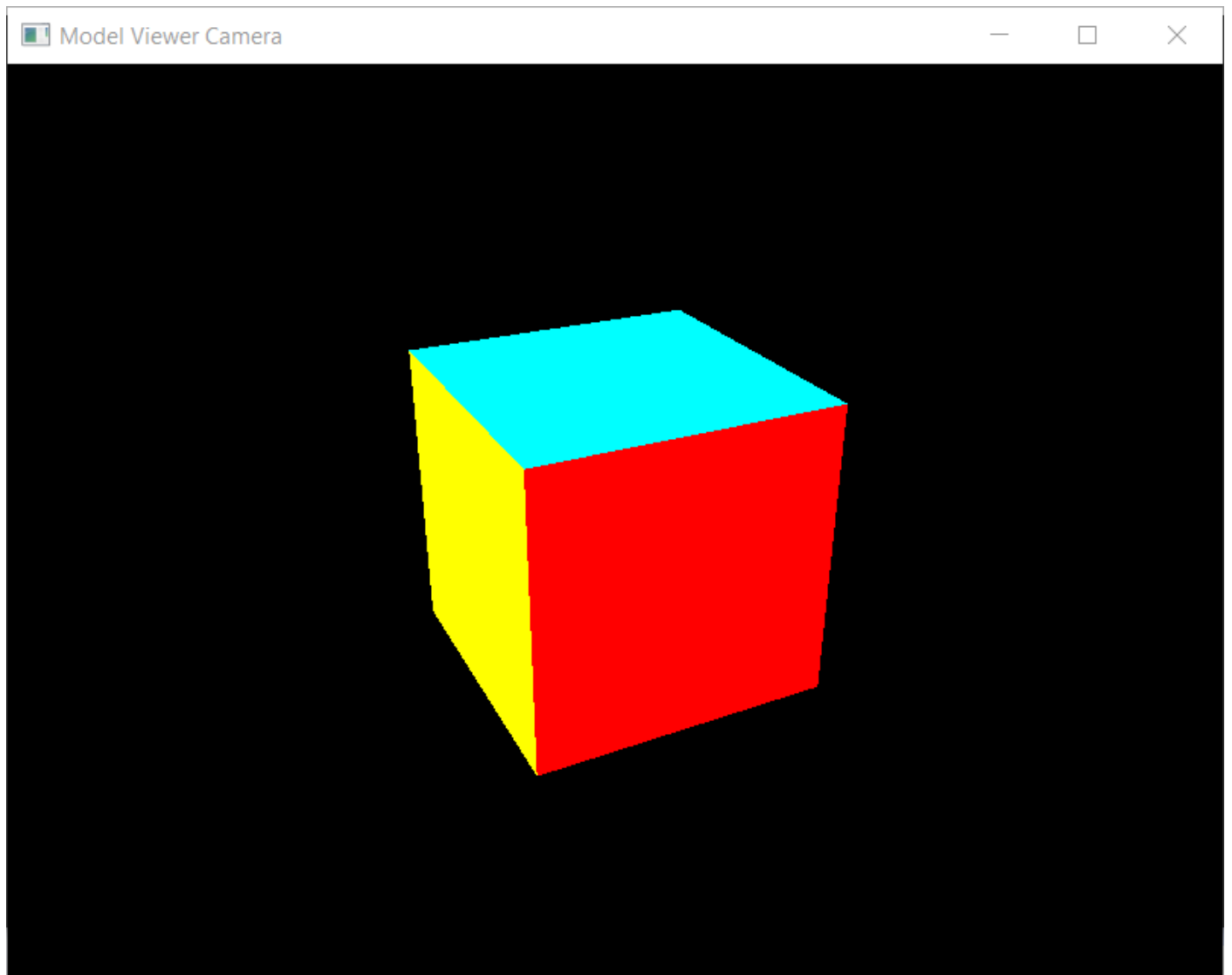
8
9 void MoveAndOrientCamera(SCamera& in, glm::vec3 target, float distance, float xoffset, float yoffset)
10 {
11     in.Yaw -= xoffset;
12     in.Pitch -= yoffset;
13
14     if (in.Pitch > 89.0f)          in.Pitch = 89.0f;
15     if (in.Pitch < -89.0f)         in.Pitch = -89.0f;
16

```

If you run the program now, you should be able to rotate in all 4 directions around the cube, and also change the distance.



Congratulations! You have implemented an intuitive camera to view your models interactively from any orientation and distance.



Demo – show your program to a member of the teaching team.

Part B Set up (Optional)

In the COMP3069 directory, create a new project called

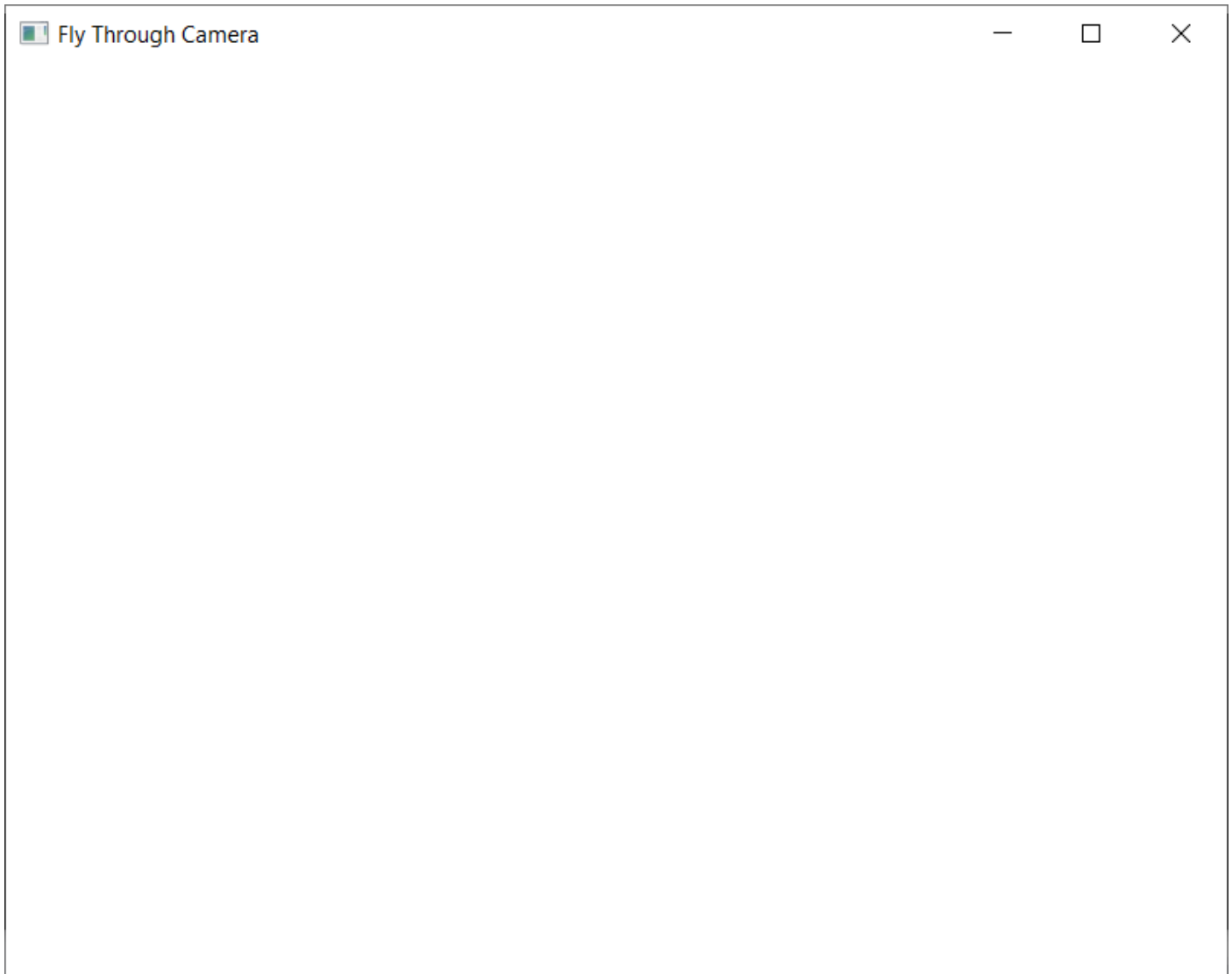
Lab7b

Download Lab7b.cpp from Moodle and copy the code to the Lab7b.cpp in your new project.

Use the vertex shader mvp.vert and fragment shader col.frag from Lab7a.

Download FlyThroughCamera.h from Moodle, copy it to your project and it to the Header Files in the Solution Explorer.

If you run the program, then the following window should appear



There is a cube but it's not visible.

Fly-Through Camera

This part of the lab is more advanced and is optional, and you will need to work things out for yourself.

You need to implement a Fly-Through Camera using some of the techniques that you learned in part A of the lab, and the definition of the camera from the lecture notes.

First, have a look at Lab7b.cpp and find the function processKeyboard(). You will see the code for detecting key presses and moving the camera position accordingly.

```
74 void processKeyboard(GLFWwindow* window)
75 {
76     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)    glfwSetWindowShouldClose(window, true);
77
78     if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)        MoveCamera(Camera, SCamera::FORWARD);
79     if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)        MoveCamera(Camera, SCamera::BACKWARD);
80     if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)        MoveCamera(Camera, SCamera::LEFT);
81     if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)        MoveCamera(Camera, SCamera::RIGHT);
82 }
```

In FlyThroughCamera.h, you will find an empty MoveCamera() function

```
6 void MoveCamera(SCamera& in, SCamera::Camera_Movement direction)
7 {
8 }
```

Also, in Lab7b.cpp, find the function processMouse(). This is a GLFW callback function which has input x and y arguments which are the mouse position on the screen. You will see the code for calculating the mouse difference in x and y, and orienting the camera accordingly.

```
88 void processMouse(GLFWwindow* window, double x, double y)
89 {
90     if (firstMouse)
91     {
92         prevMouseX = x;
93         prevMouseY = y;
94         firstMouse = false;
95     }
96
97     float dX = x - prevMouseX;
98     float dY = y - prevMouseY;
99
100     prevMouseX = x;
101     prevMouseY = y;
102
103     OrientCamera(Camera, dX, dY);
104 }
```

At the top of main, you will see the code which disables the mouse cursor and sets the cursor callback. Setting the callback means that GLFW will call the processMouse() function whenever the mouse is moved.

```
106 int main(int argc, char** argv)
107 {
108     GLFWwindow* window = CreateWindow(800, 600, "Fly Through Camera");
109     glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
110     glfwSetCursorPosCallback(window, processMouse);
111
112     gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);
113 }
```

In FlyThroughCamera.h, you will find an empty OrientCamera() function

```
10 void OrientCamera(SCamera& in, float xoffset, float yoffset)
11 {
12 }
```

Note that in the render loop, the `lookAt()` function is being called to construct the view matrix

```
156 glm::mat4 view = glm::mat4(1.f);
157 view = glm::lookAt(Camera.Position, Camera.Position + Camera.Front, Camera.Up);
158 glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"), 1, GL_FALSE, glm::value_ptr(view));
159
```

Your task is to implement the `MoveCamera()` and `OrientCamera()` functions.

When you run the program, the user should be able to change the position of the camera using the WSAD keys, and change the orientation of the camera using the mouse.

